

REPOSITORY DOCUMENTATION

Destiny Church Tees Valley — Complete Repository Documentation

Repository: Square Media Group — destinychurch

Version: 1.0.2

Last Updated: July 15, 2026

Prepared: July 18, 2026

Version: 1.0.2

Last Updated: July 15, 2026

Repository: Square Media Group — [destinychurch](#)

This document provides a comprehensive explanation of every major component, line of code purpose, architecture decisions, and how the system works from end-to-end.

Table of Contents

1. [Project Overview](#)
 2. [Architecture & Philosophy](#)
 3. [Technology Stack](#)
 4. [Directory Structure](#)
 5. [Database Schema](#)
 6. [Core Application Layers](#)
 7. [Routing & Pages](#)
 8. [Components](#)
 9. [API Routes & Server Actions](#)
 10. [Libraries & Utilities](#)
 11. [Authentication & Authorization](#)
 12. [Configuration](#)
 13. [Deployment & Performance](#)
-

Project Overview

What It Is

Destiny Church Tees Valley is a **full-stack church website platform** built by Square Media Group. It serves as the digital presence for a multi-cultural church in Stockton-on-Tees, UK, and handles:

- **Public Pages:** About, beliefs, events, contact, giving, venue hire
- **Ministry Pages:** Kids, youth, young adults, missions, connect groups
- **Media:** Sermon archive with 50+ videos, series/speaker filtering, AI-powered search
- **Member Engagement:** Prayer requests, volunteer signup, venue enquiries, connection forms
- **Admin Dashboard:** Protected area for staff to manage sermons, pages, redirects, banners, HR, jobs, training
- **Infrastructure:** Email, analytics, webhooks, caching, rate limiting

Key Visitors

- **Church members:** Watch sermons, find events, volunteer, connect groups, training
- **First-time visitors:** Plan a visit, explore beliefs, understand what to expect
- **HR/Admin staff:** Manage jobs, staff directory, leave requests, documents
- **Job applicants:** Browse positions, submit applications
- **Training participants:** Access course materials and learning paths

Non-Goals

- Real-time chat or messaging
 - Video hosting (uses YouTube)
 - Full ecommerce (giving is simplified)
 - Social network features
-

Architecture & Philosophy

Key Design Principles

1. **Server-Driven:** Use server-side rendering and server actions wherever possible; minimize client-side complexity
2. **Security First:** Row-level security (RLS) on all sensitive data; rate limiting on public APIs; Supabase service role for backend operations
3. **Edge Caching:** Long TTLs on static assets; revalidate on-demand for dynamic content
4. **Mobile First:** Responsive Tailwind CSS; tested on real devices
5. **Accessibility:** Semantic HTML, ARIA labels, keyboard navigation
6. **AI-Ready:** Structured data for future AI features (smart search, content generation)
7. **No Vendor Lock-In:** Open-source stack (Next.js, React, Tailwind, Supabase PostgreSQL)

Data Flow

```
User (Browser/Phone)
↓
Next.js App Router (Server/Client)
↓
API Routes (Node.js Serverless)
↓
Supabase PostgreSQL
↓
Responses: JSON, Server-Side HTML, Redirects, Cached Assets
```

Authentication Layers

- **Public Pages:** No auth required; cached at edge
 - **Member Features:** Supabase Auth (email/password); tokens in secure cookies
 - **Admin Dashboard:** Supabase Auth required; additional role checks via proxy
 - **API Endpoints:** Service role key (server-to-database); user tokens (client-to-server)
-

Technology Stack

Layer	Technology	Purpose
Framework	Next.js 16 (App Router)	Full-stack React with server-side rendering
Language	TypeScript	Type-safe code
Styling	Tailwind CSS v4	Utility-first CSS framework
Frontend	React 19	Component library and state management
Database	Supabase (PostgreSQL)	Relational database with RLS and real-time
Auth	Supabase Auth	User sessions and JWT tokens
Email	Resend	Transactional email (password resets, confirmations)
Storage	Supabase Storage	File uploads (HR documents, media, images)
Video	YouTube API v3	Sermon hosting and metadata
Podcasts	Buzzsprout	Podcast RSS and metadata
AI	OpenAI (<code>gpt-4.1-mini</code>)	Smart Search tool-calling chat (products, weather, directions, web search)
Payments	Stripe (Payment Element + Express Checkout)	<code>/shop</code> checkout — cards, Apple Pay, Google Pay, Link
Analytics	Vercel Analytics + SpeedInsights	Performance and visitor tracking
Deployment	Vercel	Edge functions, serverless, CDN
Testing	Playwright	E2E browser testing
Media Processing	Sharp	Uploaded images (posts, shop products, shop hero) resized/converted to WebP server-side; <code>ffmpeg-static</code> bundled but unused
Search	fuse.js + rbush	Client-side fuzzy search and spatial indexing
Icons	Phosphor + Material Symbols	Icon sets
Rich Text	TipTap (Prosemirror)	Sermon markdown, content editing
State	Zustand	Global client state (minimal usage)
Motion	Motion	Animation library
QR Codes	qrcode.react	QR code generation

Directory Structure

```
destinychurch/
├─ app/                                # Next.js App Router routes
│   ├── layout.tsx                    # Root layout (header, footer, providers)
│   ├── page.tsx                      # Home page
│   ├── globals.css                   # Tailwind + custom CSS
│   └─ robots.ts                      # robots.txt generation
│
│   # NOTE: there is no `(public-pages)` route group – every
│   # page below is a flat top-level folder directly under app/
│   ├── about/                        # About page
│   ├── accessibility/                # Reduced-motion / glass-FX preferences (client component)
│   ├── admin-login/                  # Stale-bookmark redirect → /login
│   ├── administration/               # Stale-bookmark redirect → /admin
│   ├── auth/                         # callback/ + confirm/ – Supabase OAuth & email-OTP handlers
│   ├── baptism/                     # Baptism sign-up
│   ├── beliefs/                      # Beliefs page
│   ├── bible-course/                 # The Bible Course (Bible Society) info page
│   ├── child-dedication/             # Child dedication request
│   ├── connect/                      # Connect groups page
│   ├── data-gdpr/                    # Data & GDPR policy
│   ├── dckids/                       # Destiny Kids Camp 2026 campaign page
│   ├── destiny-recovery/             # Recovery course info page
│   ├── give/                         # Giving/donations page
│   ├── help/                         # Help centre / FAQ
│   ├── kids/                        # Kids ministry
│   ├── links/                        # "Next Steps" link-in-bio style page
│   ├── live/                         # Livestream page
│   ├── login/                       # Staff sign-in
│   ├── youth/                       # Youth ministry
│   ├── young-adults/                 # Young adults ministry
│   ├── missions/                     # Missions & outreach
│   ├── privacy/                      # Privacy policy
│   ├── safeguarding/                 # Safeguarding policy
│   ├── serve/                       # Volunteer opportunities (overview)
│   ├── sermons/                     # Sermon archive (+ [id] detail)
│   ├── terms/                       # Terms of use
│   ├── training/                     # /training resource library (category → subgroup → post)
│   ├── contact/                      # Contact form
│   ├── visit/                       # Plan a visit
│   ├── new-here/                     # First-time visitor guide
│   ├── hire/                         # Venue hire enquiries
│   ├── connect-card/                 # Prayer requests & connections
│   ├── alpha/                       # Alpha course info
│   ├── volunteer/                   # Volunteer sign-up form
│   ├── whats-on/                    # Events listing
│   ├── annual-report-2025/           # Annual report campaign page
│   ├── [slug]/                      # Dynamic catchall page (posts table)
│   ├── admin/                       # Protected admin dashboard
│   │   ├── forgot-password/          # Password recovery request
│   │   ├── reset-password/           # Password reset form
│   │   ├── layout.tsx                # Admin layout (sidebar)
│   │   ├── page.tsx                 # Admin home (dashboard)
│   │   └─ banner/                    # Manage site banners
```

```

| | | | | popup/ # Manage pop-ups
| | | | | redirects/ # Manage URL redirects
| | | | | cache/ # Cache invalidation tools
| | | | | posts/ # Standalone content pages
| | | | | training/ # Training/courses management
| | | | | alpha/ # Manage Alpha course events
| | | | | bible-course/ # Manage The Bible Course events
| | | | | recovery/ # Manage Recovery course events
| | | | | featured-course/ # Choose the What's On featured course
| | | | | store/ # Shop admin (products, orders, hero)
| | | | | hr/ # HR staff features (unlinked, in progress)
| | | | | jobs/ # Job listing & application
| | | | | | page.tsx # Job list
| | | | | | [slug]/page.tsx # Job detail
| | | | | | ApplyForm.tsx # Job application form
| | | | | | actions.ts # Server actions for applying
| | | | | api/ # API routes (serverless functions) – no `public/` subfolder
| | | | | | admin/ # Admin API endpoints (gated by middleware.ts)
| | | | | | | logout/ # End admin session
| | | | | | | redirects/ # CRUD redirects
| | | | | | | popup/ # CRUD pop-ups
| | | | | | | revalidate/ # ISR cache invalidation
| | | | | | | posts/, training/, alpha-events/, featured-course/, hr/, store/, shop-hero/
| | | | | | | ...
| | | | | | chat/ # POST /api/chat – Smart Search tool-calling chat
| | | | | | youtube/ # videos/, thumbnail/[id]/, status/, live/
| | | | | | alpha-ask/, alpha-events/ # Public Alpha info endpoints
| | | | | | store/ # checkout/, checkout/bypass/ (public storefront)
| | | | | | training/ # unlock/, posts/[id]/timer/
| | | | | | health/ # smart-search/ health check
| | | | | | webhooks/ # stripe/ only – no GitHub/Vercel webhook route
| | | | | [slug]/ # Dynamic catchall (posts table)
| | | | | annual-report-2025/ # Specific campaign page
| | | | |
| | | | | components/ # React components (shared across pages)
| | | | | | ChurchHeader.tsx # Site header with nav
| | | | | | ChurchFooter.tsx # Site footer
| | | | | | Providers.tsx # Client context providers
| | | | | | CookieBanner.tsx # GDPR cookie consent
| | | | | | AnalyticsGate.tsx # Conditional analytics loading
| | | | | | SiteBanner.tsx # Announcement banner (from DB)
| | | | | | SitePopup.tsx # Modal pop-up (from DB)
| | | | | | FloatingSmartSearch.tsx # AI search widget (tool-calling chat)
| | | | | | smartSearch/ # Smart Search result cards (products, weather, maps, web)
| | | | | | LiveBanner.tsx # "WE ARE LIVE" banner bar
| | | | | | GlassBloomTracker.tsx # Glass-effect performance tracking
| | | | | | FooterGate.tsx / PerformanceGate.tsx / BannerSpacer.tsx # Layout/perf gating helpers
| | | | | | admin/ # Admin-specific components
| | | | | | | AdminSidebar.tsx # Admin nav menu
| | | | | | | AdminHeader.tsx # Admin shell header (breadcrumb + "View live site")
| | | | | | | RichTextEditor.tsx # Shared TipTap editor – posts, training posts, HR jobs,
shop products
| | | | | | training/ # Course management UI (uses RichTextEditor)
| | | | | | hr/ # HR management UI (unlinked, in progress)
| | | | | | posts/ # Content editor (uses RichTextEditor)
| | | | | | ...
| | | | | shop/ # Storefront components (ProductCard, ShopProductGrid,

```



```

ShopHero, cart, checkout)
|   ├── connect-card/           # Prayer form components
|   ├── kids/                   # Kids ministry components
|   ├── home/                   # Home page components
|   ├── visit/                  # Visit page components
|   ├── serve/                  # Volunteer components
|   ├── new-here/               # Onboarding components
|   ├── alpha/                  # Alpha course components
|   ├── AnimateIn.tsx           # Scroll-triggered animations
|   ├── ChurchSuiteEmbed.tsx    # ChurchSuite integration (events)
|   └── ...
|
|── lib/                         # Utility functions and helpers
|   ├── supabase.ts             # Supabase admin client (server-only)
|   ├── supabase-browser.ts     # Supabase client (browser)
|   ├── sermons.ts              # Sermon fetching & filtering
|   ├── youtube.ts              # YouTube API client
|   ├── smartSearch.ts           # AI search logic (parseAnswer, fallbacks)
|   └── smartSearch/tools.ts     # Smart Search tool-calling tools (products, weather, maps,
web)
|   ├── pageContent.ts          # Dynamic page editing
|   ├── posts.ts                # Dynamic posts/pages
|   ├── training.ts             # Training courses
|   ├── jobs.ts / jobs.server.ts # Job listing & applications
|   ├── hr.ts                   # HR staff operations
|   ├── shop.ts / shop.server.ts # Shop types, price helpers, published-product fetchers
|   ├── stripe.ts               # Stripe client singleton
|   ├── cart-store.ts           # Zustand basket store (localStorage-persisted)
|   └── checkout.server.ts      # Shared order/pricing logic (checkout, webhook, test
bypass)
|   ├── courses.ts              # Alpha/Recovery/Bible Course/CAP course definitions
|   ├── openaiClient.ts         # OpenAI client + SMART_SEARCH_MODEL constant
|   ├── siteKnowledge.ts        # AI search knowledge base
|   ├── serviceStatus.ts        # Feature flags
|   ├── rateLimit.ts            # Rate limiting
|   ├── loginRateLimit.ts       # Login attempt limiting
|   ├── podcast.ts              # Podcast metadata
|   ├── accessRequestEmail.ts    # Email templates
|   ├── passwordResetEmail.ts    # Email templates
|   ├── staffLogins.ts          # Staff login records
|   ├── collections.ts          # Content collections
|   ├── sermonPlayerContext.tsx  # Sermon player state
|   ├── sermonSearchContext.tsx  # Sermon search state
|   ├── cookieConsent.tsx       # Cookie preferences
|   ├── alphaSession.ts         # Alpha course sessions
|   ├── trainingAccess.ts        # Training permissions
|   ├── ai/                     # Only media-types.ts remains – the AI page-generation
|   │   └── media-types.ts       # feature (llm-client, code-generator, code-validator,
|   │                           # git-automation, page-audit-email) was removed in 7aa899d
|   │                           # alongside the old "Destiny AI" page. OpenAI is now used
|   │                           # only for Smart Search (lib/openaiClient.ts,
|
|── lib/smartSearch/)
|   ├── reserved-slugs.ts       # Protected URL paths
|   └── ...
|
|── supabase/                   # Supabase configuration
|   └── migrations/             # Database schema migrations (34 files) – selected

```

```
| |— 001_redirects.sql # URL redirect table
| |— 002_hidden_videos.sql # Hidden sermon videos (feature since removed)
| |— 003_content.sql # Site banner & page content
| |— 004_banner_type.sql # Banner types (sitewide, alpha, etc.)
| |— 006_hire_enquiries.sql # Venue hire form submissions
| |— 007_alpha_events.sql # Alpha course events
| |— 008_alpha_events_online.sql # Online/hybrid Alpha support
| |— 009_alpha_events_frequency.sql # Event recurrence
| |— 010_alpha_events_recovery_type.sql # Recovery program
| |— 20260329175837_add_subject_to_contact_messages.sql # Contact form subject field
| |— 20260502_create_site_popup.sql # Modal pop-ups
| |— 20260507_create_builder_media_bucket.sql # AI page builder media (builder since
removed)
```

```
|— 20260531_hr.sql          # HR staff, leave, reviews, documents
|— 20260606_jobs.sql        # Job listings & applications
|— 20260608_service_status.sql # Feature flags
|— 20260614_staff_logins.sql # Staff login audit trail
|— 20260616_training.sql, 20260616_0f2,3,4 *.sql # Training
```

```
| └─ 20260618_posts.sql      # Standalone posts (`/[slug]` catch-all)
| └─ 20260708_shop.sql      # Shop: products, variants, orders, items
| └─ 20260708_shop_product_fit.sql # products.fit (male/female/unisex/kids)
| └─ 20260710_shop_hero.sql # Editable auto-rotating /shop hero slides
| └─ 20260711_02..07_*.sql  # RLS/security-advisor fixes: shop product type: page
```

```
| └─ 20260711_rls_harden_base_tables.sql # RLS hardening pass across base tables
| └─ 20260712_alpha_events_bible_course_type.sql # The Bible Course
| └─ 20260712_02_featured_course.sql # Featured course (What's On)
```

```
| contexts/           # React context definitions
| content/           # Static content (markdown, text)
| docs/              # Additional documentation
| public/            # Static files
|   | img/           # Church logos, backgrounds
|   | og/            # Open Graph images (social share)
|   | fonts/         # Custom fonts
|   |
```

```
|— .claude/                # Claude Code settings
|— .vscode/               # VS Code workspace settings
|— next.config.ts         # Next.js configuration
|— tailwind.config.ts     # Tailwind CSS customization
|— playwright.config.ts   # E2E test configuration
|— eslint.config.mjs      # ESLint rules
|— postcss.config.mjs     # PostCSS plugins
```

```
|— tsconfig.json      # TypeScript compiler options
|— package.json       # Node dependencies and scripts
|— package-lock.json  # Locked versions
|— .gitignore         # Git exclusions
|— CLAUDE.md          # Project instructions
|— README.md          # Public project README
|— REPOSITORY_DOCUMENTATION.md # This file
```

Database Schema

Overview

The database uses Supabase (managed PostgreSQL) with Row-Level Security (RLS) for sensitive data. Public tables are readable via the anonymous key; member-only and admin tables use the service role key (server-to-database communication).

Tables

1. redirects

Purpose: URL redirect management (e.g., `/prayer-request` → `/connect-card`)

```
CREATE TABLE redirects (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  slug text UNIQUE NOT NULL,          -- /short-path  
  target_url text NOT NULL,           -- https://example.com or /absolute/path  
  label text,                         -- Display name (unused currently)  
  active boolean DEFAULT true,        -- Toggle visibility  
  created_at timestamptz DEFAULT now()  
);  
  
-- RLS: Public read access to active redirects; service role has full access
```

Used By: - `app/api/[slug]` or Next.js `redirects()` config during build - Admin dashboard to manage redirects

3. site_banner

Purpose: Sitewide announcement banners (top of every page)

```
CREATE TABLE site_banner (  
  id uuid PRIMARY KEY,  
  message text NOT NULL,              -- Banner text (HTML allowed)  
  active boolean DEFAULT false,       -- Toggle on/off  
  type text,                          -- 'sitewide', 'alpha', 'youth_alpha', 'recovery',  
  'bible_course'  
  link text,                          -- Optional URL for banner  
  link_text text,                     -- CTA button text  
  created_at timestamptz DEFAULT now(),  
  updated_at timestamptz  
);  
  
-- Types reference alpha_events for dynamic data (e.g., next Alpha start date)  
-- RLS: Service role only
```

Used By: - `app/layout.tsx` fetches active banner on every request - Admin dashboard to edit banners

4. page_content

Purpose: Key-value store for editable site content

```
CREATE TABLE page_content (  
  page text NOT NULL,          -- 'give', 'general', etc.  
  key text NOT NULL,          -- 'account_name', 'service_time'  
  value text DEFAULT '',      -- Content value  
  updated_at timestamptz DEFAULT now(),  
  PRIMARY KEY (page, key)  
);  
  
-- Example rows:  
-- ('give', 'account_name', 'Destiny Church Tees Valley')  
-- ('give', 'sort_code', '08-92-99')  
-- ('general', 'service_time', '11am')  
-- RLS: Service role only
```

Used By: - [lib/pageContent.ts](#) to fetch dynamic content - Admin dashboard to edit giving details, service times, etc.

5. contact_messages

Purpose: Messages from the public contact form

```
CREATE TABLE contact_messages (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  name text NOT NULL,  
  email text NOT NULL,  
  subject text,                -- Added later (migration 20260329)  
  message text NOT NULL,  
  created_at timestamptz DEFAULT now()  
);  
  
-- RLS: Public insert (form submission); service role read
```

Used By: - [app/contact/actions.ts](#) (server action) inserts submissions - Admin dashboard to view inquiries

6. hire_enquiries

Purpose: Venue hire request submissions

```
CREATE TABLE hire_enquiries (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  email text NOT NULL,
  phone text NOT NULL,
  date_needed date NOT NULL,
  event_description text NOT NULL,
  approx_guests integer,
  created_at timestamptz DEFAULT now()
);

-- RLS: Public insert; service role read
```

Used By: - `app/hire/actions.ts` (server action) inserts submissions - Admin dashboard to view inquiries

7. alpha_events

Purpose: Alpha course event scheduling and details

```
CREATE TABLE alpha_events (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  type text NOT NULL,                -- 'alpha', 'youth_alpha', 'recovery', 'bible_course'
  active boolean DEFAULT true,
  start_date date NOT NULL,
  signup_url text,                  -- External booking link
  format text,                     -- 'in_person', 'online', 'hybrid'
  location text,                   -- 'Destiny Centre', etc.
  meeting_platform text,           -- 'Zoom', 'Teams', etc. (if online)
  frequency text DEFAULT 'weekly', -- 'weekly', 'fortnightly'
  custom_interval_days integer,    -- Override frequency
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz
);

-- RLS: Public read; service role write
```

Used By: - `app/layout.tsx` fetches to populate site-wide banner - `/alpha`, `/destiny-recovery` and `/bible-course` pages display their next upcoming event - Admin (`/admin/alpha`, `/admin/recovery`, `/admin/bible-course`) to manage event dates and URLs

The `bible_course` type is shared infrastructure for The Bible Course (Bible Society) — it reuses this table and the `/api/admin/alpha-events` routes rather than adding new ones. Added by migration `20260712_alpha_events_bible_course_type.sql`.

7b. featured_course

Purpose: Singleton setting for which course headlines the What's On "Courses" section.

```
CREATE TABLE featured_course (
  id integer PRIMARY KEY DEFAULT 1,          -- always 1 (singleton)
  course_id text NOT NULL DEFAULT 'bible_course'
  CHECK (course_id IN ('bible_course','cap','alpha','recovery')),
  updated_at timestamptz DEFAULT now()
);
-- RLS: enabled, deny-all ("service only"); server routes use the service key.
```

The four courses are defined in `lib/courses.ts` (each with a grid-card and a full-width "featured" config). The featured course renders as the wide banner; the other three render as cards, so all four are always visible. Chosen at `/admin/featured-course`.

Used By: - `app/whats-on/page.tsx` reads it and passes `featuredId` to `CoursesSection` - `app/api/admin/featured-course` (GET/PUT) — PUT revalidates `/whats-on` - Migration `20260712_02_featured_course.sql`

8. site_popup

Purpose: Modal pop-ups that appear once per session/visitor

```
CREATE TABLE site_popup (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  active boolean DEFAULT false,
  title text,
  body text,                                -- Supports markdown/HTML
  cta_text text,                             -- Button label
  cta_link text,                             -- Button URL
  image_url text,                           -- Popup image
  show_once boolean DEFAULT true,           -- Show only once per browser session
  updated_at timestamptz DEFAULT now()
);
-- RLS: Service role only
```

Used By: - `app/layout.tsx` fetches active pop-up - Displayed by `SitePopup.tsx` component - Admin dashboard to manage pop-ups

10. hr_staff

Purpose: Employee/volunteer directory

```

CREATE TABLE hr_staff (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  first_name text NOT NULL,
  last_name text NOT NULL,
  email text,
  phone text,
  job_title text,
  department text,
  employment_type text NOT NULL DEFAULT 'full_time'
    CHECK (employment_type IN ('full_time', 'part_time', 'volunteer', 'contractor')),
  status text NOT NULL DEFAULT 'active'
    CHECK (status IN ('active', 'on_leave', 'left')),
  start_date date,
  end_date date,
  annual_leave_entitlement numeric DEFAULT 0,
  notes text,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

-- Trigger: hr_staff_updated_at auto-updates `updated_at` on every change
-- RLS: Service role only (accessed via proxy requiring auth)

```

Used By: - HR admin to manage staff directory - </admin/hr> dashboard

11. hr_leave_requests

Purpose: Time-off and leave requests

```

CREATE TABLE hr_leave_requests (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  staff_id uuid NOT NULL REFERENCES hr_staff(id) ON DELETE CASCADE,
  type text NOT NULL DEFAULT 'holiday'
    CHECK (type IN ('holiday', 'sick', 'other')),
  start_date date NOT NULL,
  end_date date NOT NULL,
  days numeric DEFAULT 0,          -- Calculated number of working days
  reason text,
  status text NOT NULL DEFAULT 'pending'
    CHECK (status IN ('pending', 'approved', 'rejected')),
  reviewed_by text,              -- Name of approver
  reviewed_at timestamptz,
  created_at timestamptz DEFAULT now()
);

-- RLS: Service role only
-- Indexes: staff_id, status

```

Used By: - HR staff to request/approve leave - Admin dashboard for leave tracking

12. hr_documents

Purpose: HR document library (contracts, policies, payslips)

```
CREATE TABLE hr_documents (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  staff_id uuid REFERENCES hr_staff(id) ON DELETE CASCADE, -- NULL = org-wide  
  title text NOT NULL,  
  category text NOT NULL DEFAULT 'other'  
  CHECK (category IN ('contract', 'policy', 'payslip', 'other')),  
  file_path text NOT NULL, -- Supabase Storage path  
  file_name text NOT NULL,  
  mime_type text,  
  size_bytes bigint,  
  uploaded_by text,  
  created_at timestamptz DEFAULT now()  
);  
  
-- RLS: Service role only  
-- Indexes: staff_id  
-- Files stored in 'hr-documents' private bucket (signed URLs only)
```

Used By: - Admin to upload and organize HR documents - Staff to download their documents via signed URLs

13. hr_reviews

Purpose: Appraisals and 1-to-1 meeting records

```
CREATE TABLE hr_reviews (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  staff_id uuid NOT NULL REFERENCES hr_staff(id) ON DELETE CASCADE,  
  review_date date NOT NULL,  
  type text NOT NULL DEFAULT 'one_to_one'  
  CHECK (type IN ('appraisal', 'one_to_one')),  
  reviewer text, -- Name of reviewer  
  summary text, -- Notes/outcomes  
  next_review_date date, -- Scheduled follow-up  
  created_at timestamptz DEFAULT now()  
);  
  
-- RLS: Service role only  
-- Indexes: staff_id
```

Used By: - Admin to log reviews - HR dashboard to track review schedule

14. jobs

Purpose: Job listings and applications

```

CREATE TABLE jobs (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  title text NOT NULL,           -- Job title
  slug text UNIQUE NOT NULL,     -- URL slug
  description text NOT NULL,     -- Full job description (markdown)
  department text,               -- 'Worship', 'Admin', etc.
  employment_type text,         -- 'full_time', 'part_time'
  salary_range text,            -- Free text (e.g., "£25,000-£30,000")
  closing_date date,            -- Application deadline
  active boolean DEFAULT true,   -- Hide expired jobs
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

-- RLS: Public read (active only); service role write
-- Indexes: slug, active

```

Used By: - </jobs> page to list open positions - [/jobs/\[slug\]](/jobs/[slug]) for job detail pages - Admin to create/edit job postings

15. job_applications

Purpose: Submitted job applications

```

CREATE TABLE job_applications (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id uuid NOT NULL REFERENCES jobs(id) ON DELETE CASCADE,
  name text NOT NULL,
  email text NOT NULL,
  phone text NOT NULL,
  cv_url text,                  -- Uploaded CV (signed URL)
  cover_letter text,            -- Freeform text
  status text DEFAULT 'pending'
  CHECK (status IN ('pending', 'reviewing', 'rejected', 'offered')),
  created_at timestamptz DEFAULT now()
);

-- RLS: Public insert (application form); service role read
-- Indexes: job_id, status

```

Used By: - Job application form (<app/jobs/ApplyForm.tsx>) - Admin dashboard to review applications

16. service_status

Purpose: Feature flags for experimental/beta features

```
CREATE TABLE service_status (
  service_name text PRIMARY KEY,      -- 'smart_search', etc.
  enabled boolean DEFAULT false,
  updated_at timestampz DEFAULT now()
);

-- RLS: Public read; service role write
```

Used By: - `lib/serviceStatus.ts` to check if features are active - Admin to toggle features on/off without deploying

17. posts

Purpose: Generic published pages served by the `/[slug]` catch-all route (freeform site pages outside the main nav)

```
CREATE TABLE posts (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  title text NOT NULL,
  slug text UNIQUE NOT NULL,
  body text,
  is_published boolean NOT NULL DEFAULT false,
  created_at timestampz DEFAULT now(),
  updated_at timestampz DEFAULT now()
);

-- RLS: Service role only; public read happens server-side via lib/posts.server.ts
```

Used By: - `lib/posts.server.ts` (`getPublishedPostBySlug`) for the public `/[slug]` catch-all - `app/api/admin/posts` CRUD, admin dashboard to write pages

17b. training_categories / training_subgroups / training_folders / training_posts

Purpose: The `/training` member resource library — a category → sub-group (optionally password-protected) → folder → post hierarchy.

```

CREATE TABLE training_categories (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL, slug text UNIQUE NOT NULL, description text, icon text,
  sort_order integer NOT NULL DEFAULT 0,
  is_published boolean NOT NULL DEFAULT true,
  created_at timestamptz DEFAULT now(), updated_at timestamptz DEFAULT now()
);

CREATE TABLE training_subgroups (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  category_id uuid NOT NULL REFERENCES training_categories(id),
  name text NOT NULL, slug text NOT NULL, description text,
  password_hash text,          -- NULL = no password gate; never returned to clients,
only has_password
  sort_order integer NOT NULL DEFAULT 0,
  is_published boolean NOT NULL DEFAULT true,
  created_at timestamptz DEFAULT now(), updated_at timestamptz DEFAULT now()
);

CREATE TABLE training_folders (
  id uuid PRIMARY KEY DEFAULT extensions.uuid_generate_v4(),
  subgroup_id uuid NOT NULL REFERENCES training_subgroups(id),
  name text NOT NULL, sort_order integer NOT NULL DEFAULT 0,
  created_at timestamptz DEFAULT now(), updated_at timestamptz DEFAULT now()
);

CREATE TABLE training_posts (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  subgroup_id uuid NOT NULL REFERENCES training_subgroups(id),
  folder_id uuid REFERENCES training_folders(id),  -- NULL = ungrouped within the subgroup
  title text NOT NULL, slug text NOT NULL, summary text, body text,
  min_read_seconds integer NOT NULL DEFAULT 0,
  sort_order integer NOT NULL DEFAULT 0,
  is_published boolean NOT NULL DEFAULT false,
  created_at timestamptz DEFAULT now(), updated_at timestamptz DEFAULT now()
);

-- RLS: Service role only. Public read of published rows happens server-side
-- via lib/training.server.ts; password_hash is never selected into client-facing shapes.

```

Used By: - `lib/training.server.ts` (`getTrainingTree`) for the public `/training` tree -
`app/api/training/unlock` to verify a sub-group password -
`app/api/admin/training/{categories,subgroups,folders,posts}` CRUD, admin dashboard

Row-Level Security (RLS) Strategy

18. products / product_variants / orders / order_items (Shop)

Purpose: The Stripe-powered store (`/shop`), replacing the old WooCommerce site. Physical apparel with size + colour variants and per-variant stock; collection-only fulfilment. Prices are integer **pennies** (GBP).

Migration: `supabase/migrations/20260708_shop.sql` , extended by `20260708_shop_product_fit.sql` (`fit`) and `20260711_05_shop_product_type.sql` (`product_type`).

```

CREATE TABLE products (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  slug text UNIQUE NOT NULL,
  name text NOT NULL,
  description text,
  base_price_pennies integer NOT NULL DEFAULT 0,
  category text,
  fit text NOT NULL DEFAULT 'unisex'          -- 'male'|'female'|'unisex'|'kids' – drives size
chart + storefront label
  CHECK (fit IN ('male','female','unisex','kids')),
  product_type text NOT NULL DEFAULT 'clothing' -- 'clothing'|'books'|'other' – colour/size
matrix vs. format vs. free-form
  CHECK (product_type IN ('clothing','books','other')),
  images jsonb NOT NULL DEFAULT '[]',    -- [{ url, path, alt }]
  is_published boolean NOT NULL DEFAULT false,
  is_featured boolean NOT NULL DEFAULT false,
  sort_order integer NOT NULL DEFAULT 0,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

CREATE TABLE product_variants (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id uuid REFERENCES products(id) ON DELETE CASCADE,
  size text, color text, color_hex text, sku text,
  price_pennies integer,                -- null → inherit product base price
  stock integer NOT NULL DEFAULT 0,
  is_active boolean NOT NULL DEFAULT true,
  sort_order integer NOT NULL DEFAULT 0,
  UNIQUE (product_id, size, color)
);

CREATE TABLE orders (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  order_number text UNIQUE NOT NULL,    -- human-friendly, e.g. DT-7F3K9Q2X
  stripe_payment_intent_id text UNIQUE,
  customer_name text NOT NULL, customer_email text NOT NULL, customer_phone text,
  notes text,
  status text NOT NULL DEFAULT 'pending', -- pending|paid|fulfilled|cancelled|refunded
  fulfillment_method text NOT NULL DEFAULT 'collection',
  subtotal_pennies integer, total_pennies integer, currency text DEFAULT 'gbp',
  paid_at timestamptz, created_at timestamptz DEFAULT now(), updated_at timestamptz DEFAULT
now()
);

CREATE TABLE order_items (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id uuid REFERENCES orders(id) ON DELETE CASCADE,
  product_id uuid, variant_id uuid,    -- snapshot fields survive product deletion
  product_name text NOT NULL, size text, color text,
  unit_price_pennies integer NOT NULL, quantity integer NOT NULL DEFAULT 1
);
-- RLS: single "service only" policy on all four (like jobs). Public storefront
-- reads via lib/shop.server.ts; checkout/webhook write via the service role.
-- Storage: public `product-images` bucket for product photos (WebP).

```

Used By: `/shop` storefront, `/admin/store` CRUD, `POST /api/store/checkout`, `POST /api/webhooks/stripe`. `description` is HTML written via the shared `RichTextEditor` (same component used for posts/training) — the admin editor no longer has a markdown Write/Preview toggle, and the public product page renders it with `dangerouslySetInnerHTML` (no markdown fallback).

19. shop_hero_slides (Shop hero)

Purpose: Dynamic, admin-editable hero at the top of `/shop`. Multiple active slides auto-rotate (crossfade) on the storefront; with no active slides the shop shows its static "The Destiny Store" masthead. Migration: `supabase/migrations/20260710_shop_hero.sql`.

```
CREATE TABLE shop_hero_slides (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  active boolean NOT NULL DEFAULT true,  
  heading text, subheading text,  
  cta_text text, cta_link text,  
  image_url text, image_path text,      -- storage path in shop-hero-images bucket  
  sort_order integer NOT NULL DEFAULT 0,  
  created_at timestamptz DEFAULT now(), updated_at timestamptz DEFAULT now()  
);  
-- RLS: single "service only" policy (like the shop tables). Public storefront  
-- reads active slides via lib/shop.server.ts → getActiveShopHeroSlides().  
-- Storage: public `shop-hero-images` bucket for hero backgrounds (WebP).
```

Used By: `/shop` storefront (`components/shop/ShopHero.tsx`), `/admin/store/hero` CRUD.

All tables have RLS enabled. Access rules:

Table	Public Read	Authenticated Read	Service Role	Purpose
redirects	Yes	-	Yes	Public navigation
site_banner	Yes	-	Yes	Show on every page
page_content	-	-	Yes	Protect sensitive values
contact_messages	-	-	Yes	Protect submissions
hire_enquiries	-	-	Yes	Protect submissions
alpha_events	Yes	-	Yes	Public event dates
featured_course	-	-	Yes	Featured course setting (read via server component)
site_popup	-	-	Yes	Protect pop-up content
hr_* (staff, leave, reviews, docs)	-	-	Yes	Sensitive HR data
jobs	Yes	-	Yes	Public listings
job_applications	-	-	Yes	Protect applications
service_status	Yes	-	Yes	Feature flags
posts	-	-	Yes	Freeform pages (public read via server components)
training_categories / training_subgroups / training_folders / training_posts	-	-	Yes	/training resource library (public read via server components; sub-group passwords hashed)
products / product_variants	-	-	Yes	Shop catalogue (public read via server components)
orders / order_items	-	-	Yes	Store orders (written by Stripe webhook)
shop_hero_slides	-	-	Yes	Editable /shop hero (public read via server components)

Key Point: No table uses authenticated user RLS. All member-facing features use API proxy routes that enforce authentication in application code, then access the database with the service role key. This gives finer control and better error messages.

Core Application Layers

Layer 1: Root Layout (`app/layout.tsx`)

The root layout wraps every page in the application. It:

1. **Defines metadata & SEO** — Title templates, description, Open Graph images for social share
2. **Fetches server data** — Banner, pop-up, feature flags (called on every request)
3. **Renders custom SVG filter** — A sophisticated glass refraction effect used for visual polish
4. **Wraps in providers** — Context providers for auth, theme, analytics
5. **Loads fonts** — Roboto, Anton, Playfair Display from Google Fonts
6. **Sets up structure** — Header, main content area, footer

Key Code:


```

// Fetch active banner (shown at top of site)
async function getActiveBanner() {
  const supabase = createServiceClient();
  const { data: rows } = await supabase
    .from("site_banner")
    .select("active, message, type, link, link_text")
    .eq("active", true);

  // If multiple banners, prioritize sitewide > event banners
  const sitewide = banners.find((b) => b.type === "sitewide");
  if (sitewide) return sitewide;

  // Otherwise return first active event banner (Alpha, Youth Alpha, Recovery)
  // ...
}

// Fetch active pop-up (modal overlay)
async function getActivePopup() {
  const supabase = createServiceClient();
  const { data } = await supabase
    .from("site_popup")
    .select("active, title, body, cta_text, cta_link, image_url, show_once")
    .eq("active", true)
    .limit(1)
    .maybeSingle(); // Returns null if no results
  return data ?? null;
}

// Render root HTML (simplified – the real tree also renders LiveBanner,
// GlassBloomTracker, FooterGate/PerformanceGate wrappers, and passes a
// server-seeded `live` status into Providers; see components/ for each)
export default async function RootLayout({ children }) {
  const banner = await getActiveBanner();
  const popup = await getActivePopup();
  const smartSearchEnabled = await isSmartSearchEnabled();

  return (
    <html>
      <body className="...">
        <svg aria-hidden>
          {/* Glass refraction filter (SVG filters for visual effects) */}
        </svg>

        <Providers banner={banner}>
          <CookieBanner />
          <div className="flex flex-col">
            <ChurchHeader />
            <main>{children}</main>
            <ChurchFooter />
          </div>
          <AnalyticsGate /> {/* Conditionally load Vercel Analytics */}
          <SitePopup popup={popup} />
          {smartSearchEnabled && <FloatingSmartSearch />} {/* AI chat widget */}
        </Providers>

        <SpeedInsights /> {/* Vercel performance monitoring */}
      </body>
    </html>
  );
}

```

```

    </body>
  </html>
);
}

```

SVG Glass Refraction Filter Explanation:

The `<filter id="glass-refract">` is a sophisticated visual effect:

1. **feGaussianBlur** — Blur the alpha channel to create a soft edge ramp
2. **feOffset** — Offset the blurred version left/right and up/down to detect edges
3. **feComposite** — Subtract offsets to create signed X/Y displacement vectors
4. **feDisplacementMap** — Apply the displacement at three scales (42/48/54) to separate color channels slightly
5. **feBlend** — Recombine RGB channels; the offset makes colors separate at edges (chromatic dispersion)

Result: Elements with `.glass-refract` backdrop-filter get a thick glass effect with color separation at edges.

Layer 2: Providers (`components/Providers.tsx`)

Wraps children with React context providers:

- **Supabase Client Provider** — Browser auth token management
- **Theme Provider** — Dark/light mode
- **Query Client** (if using) — API caching
- **Toast Provider** (`components/ToastProvider.tsx`) — Passive notifications via `useToast()` (`success` / `error` / `info`)
- **Dialog Provider** (`components/DialogProvider.tsx`) — Styled, promise-based modal dialogs via `useDialog()`

Dialogs & notifications — no native browser dialogs

Native `alert()`, `confirm()`, and `window.prompt()` are **banned** in this codebase — they look untrustworthy, can't be styled, and freeze the tab. Use the in-app equivalents instead:

- **Passive message** → `useToast()` from `components/ToastProvider.tsx`: `toast.error("...")`, `toast.success("...")`, `toast.info("...")`.
- **Yes/No decision** → `useDialog().confirm(opts)` → `Promise<boolean>`. Renders a centered, blocking styled modal. Options: `{ title?, message, confirmLabel?, cancelLabel?, tone? }` where `tone: "danger"` gives a red confirm button for destructive actions. Pattern: `if (!(await confirm({ message: "Delete this?", tone: "danger", confirmLabel: "Delete" }))) return;` (the enclosing handler must be `async`).
- **Free-text input** → `useDialog().prompt(opts)` → `Promise<string | null>` (`null` = cancelled, matching native `prompt`). Options: `{ title?, message?, placeholder?, defaultValue?, confirmLabel? }`.

Both dialog helpers reuse the visual conventions from `components/admin/hr/HrUI.tsx` (backdrop + rounded card + shared button/input classes). Escape and backdrop-click cancel.

Layer 3: Header & Navigation (`components/ChurchHeader.tsx`)

Rendered on every page (server component with Suspense).

- **Logo** — Clickable link to home
 - **Navigation menu** — Links to all main pages
 - **Mobile menu** — Hamburger on small screens
 - **Search bar** — Filters sermons/content (client-side with fuse.js)
 - **Auth indicator** — Login/logout buttons
-

Layer 4: Main Content (Route-Specific Components)

Each page route (`app/*/page.tsx`) renders page-specific content. Examples:

`/app/sermons/page.tsx` — Sermon Archive

- Fetches YouTube videos via `lib/youtube.ts`
- Filters by series/speaker/date using client-side state
- Displays video grid with thumbnails and metadata
- Lazy-loads videos on scroll

`/app/[slug]/page.tsx` — Dynamic Catchall

- Looks up `slug` via `getPublishedPostBySlug()` (`lib/posts.server.ts`) against the `posts` table — there is no separate `dynamic_pages` table
 - Renders the post's HTML `body` with `dangerouslySetInnerHTML`
 - Falls back to 404 (`notFound()`) if no published post matches
-

Layer 5: Footer (`components/ChurchFooter.tsx`)

Displayed on every page:

- **Contact info** — Address, phone, email
 - **Social links** — YouTube, Facebook, Instagram
 - **Quick links** — Common pages
 - **Copyright** — Auto-updates year
 - **Accessibility statement**
-

Routing & Pages

Public Pages (No Auth Required)

Route	File	Purpose
/	app/page.tsx	Home page — hero, featured sermon, CTAs
/about	app/about/page.tsx	About church, team, vision, mission
/beliefs	app/beliefs/page.tsx	Statement of faith, doctrine
/sermons	app/sermons/page.tsx	Archive of all sermon videos, searchable
/sermons/[id]	app/sermons/[id]/page.tsx	Individual sermon detail, video embed, transcript
/live	app/live/page.tsx	Livestream page — custom glass player when live, offline state otherwise
/contact	app/contact/page.tsx	Contact form, address, hours
/give	app/give/page.tsx	Giving info — bank details, online giving
/shop	app/shop/page.tsx	Store front — published products grid with category filter chips (<code>ShopProductGrid</code>), editorial <code>/links</code> style
/shop/[slug]	app/shop/[slug]/page.tsx	Product detail — gallery, size/colour variant picker, add to basket
/shop/cart	app/shop/cart/page.tsx	Basket (client, zustand + localStorage)
/shop/checkout	app/shop/checkout/page.tsx	Contact details + embedded Stripe Payment Element
/shop/checkout/success	app/shop/checkout/success/page.tsx	Order confirmation; clears the basket
Also served at <code>shop.destinytees.uk/*</code>	<code>next.config.ts</code> <code>rewrites()</code>	Legacy subdomain host-rewrites to <code>/shop/*</code>
/visit	app/visit/page.tsx	First-time visitor guide, parking, service times
/new-here	app/new-here/page.tsx	Onboarding for new members

Route	File	Purpose
/hire	app/hire/page.tsx	Venue hire enquiry form
/serve	app/serve/page.tsx	Volunteer opportunities
/connect	app/connect/page.tsx	Connect groups (small groups)
/kids	app/kids/page.tsx	Kids ministry (ages 0-11)
/youth	app/youth/page.tsx	Youth ministry (ages 11-18)
/young-adults	app/young-adults/page.tsx	Young adults (18-30s)
/missions	app/missions/page.tsx	Mission partners, outreach
/alpha	app/alpha/page.tsx	Alpha course info, next event
/bible-course	app/bible-course/page.tsx	The Bible Course (Bible Society), next event
/whats-on	app/whats-on/page.tsx	Events listing (ChurchSuite embed)
/connect-card	app/connect-card/page.tsx	Prayer requests, connection form
/jobs	app/jobs/page.tsx	Job listings
/jobs/[slug]	app/jobs/[slug]/page.tsx	Job detail page
/training	app/training/page.tsx	/training resource library — category → subgroup (optional password) → post
/baptism	app/baptism/page.tsx	Baptism sign-up
/child-dedication	app/child-dedication/page.tsx	Child dedication request
/volunteer	app/volunteer/page.tsx	Volunteer sign-up form
/help	app/help/page.tsx	Help centre / FAQ
/links	app/links/page.tsx	"Next Steps" link-in-bio style page
/destiny-recovery	app/destiny-recovery/page.tsx	Recovery course info page
/dckids	app/dckids/page.tsx	Destiny Kids Camp 2026 campaign page
/accessibility	app/accessibility/page.tsx	Reduced-motion / glass-FX preferences (client component)
/privacy	app/privacy/page.tsx	Privacy policy
/terms	app/terms/page.tsx	Terms of use
/safeguarding	app/safeguarding/page.tsx	Safeguarding policy

Route	File	Purpose
<code>/data-gdpr</code>	<code>app/data-gdpr/page.tsx</code>	Data & GDPR policy
<code>/annual-report-2025</code>	<code>app/annual-report-2025/page.tsx</code>	Annual report campaign page
<code>/admin-login</code> , <code>/administration</code>	<code>app/admin-login/page.tsx</code> , <code>app/administration/page.tsx</code>	Stale-bookmark redirects to <code>/login</code> and <code>/admin</code>
<code>/auth/callback</code> , <code>/auth/confirm</code>	<code>app/auth/callback/route.ts</code> , <code>app/auth/confirm/route.ts</code>	Supabase OAuth callback and email-OTP verification route handlers
<code>/[slug]</code>	<code>app/[slug]/page.tsx</code>	Dynamic catchall — looks up a published row in the <code>posts</code> table

Admin Pages (Auth Required, Checked in `middleware.ts`)

All admin/staff features live under a single `/admin` prefix with one login at `/login` (no per-section roles — any authenticated staff account has full access). `/admin/hr` is built but intentionally unlinked from any nav (not yet launched).

Route	File	Purpose
/login	app/login/page.tsx	Staff sign-in
/admin/forgot-password	app/admin/forgot-password/page.tsx	Password reset request
/admin/reset-password	app/admin/reset-password/page.tsx	Password reset form
/admin	app/admin/page.tsx	Admin dashboard home
/admin/banner	app/admin/banner/page.tsx	Manage site banners
/admin/popup	app/admin/popup/page.tsx	Manage pop-ups
/admin/redirects	app/admin/redirects/page.tsx	Manage URL redirects
/admin/cache	app/admin/cache/page.tsx	Invalidate ISR cache
/admin/posts	app/admin/posts/page.tsx	Standalone content pages
/admin/training	app/admin/training/page.tsx	Training categories → subgroups → posts
/admin/alpha	app/admin/alpha/page.tsx	Manage Alpha course events
/admin/bible-course	app/admin/bible-course/page.tsx	Manage The Bible Course events
/admin/recovery	app/admin/recovery/page.tsx	Manage Recovery course events
/admin/featured-course	app/admin/featured-course/page.tsx	Choose the What's On featured course
/admin/hr	app/admin/hr/page.tsx	HR dashboard (staff, leave, jobs, documents, reviews) — unlinked, in progress
/admin/store	app/admin/store/page.tsx	Store — product list
/admin/store/products/new	app/admin/store/products/new/page.tsx	Create a product (name → editor)
/admin/store/products/[id]	app/admin/store/products/[id]/page.tsx	Product editor — details, photos, size/colour variants, stock
/admin/store/hero	app/admin/store/hero/page.tsx	Shop hero slides — add/edit/reorder rotating hero
/admin/store/orders	app/admin/store/orders/page.tsx	Orders list
/admin/store/orders/[id]	app/admin/store/orders/[id]/page.tsx	Order detail — mark fulfilled/cancelled/refunded

Components

Global Components

ChurchHeader.tsx

- **What:** Site navigation header
- **Props:** None (uses client context for mobile menu state)
- **Behavior:**
 - Desktop: Horizontal menu bar with search
 - Mobile: Hamburger menu (drawer slides from left)
 - Active route highlighting
 - Search filters sermons by title/speaker (client-side fuse.js)

ChurchFooter.tsx

- **What:** Sitewide footer
- **Displays:** Contact info, quick links, social, copyright
- **Dynamic content:** Address and hours from `page_content` table

CookieBanner.tsx

- **What:** GDPR cookie consent banner
- **Behavior:**
 - Shows once per session
 - User can accept all / essential only
 - Preference stored in localStorage
 - Disables analytics if rejected

AnalyticsGate.tsx

- **What:** Conditional analytics loading
- **Logic:** Only loads Vercel Analytics if cookie consent given
- **Files:** `_document.js` or inline script tag

SiteBanner.tsx

- **What:** Top-of-page announcement banner
- **Data:** `site_banner` table (fetched in root layout)
- **Features:**
 - Multiple banner types (sitewide, alpha events, recovery)
 - Optional CTA link
 - Can be dismissed by user (session storage)

SitePopup.tsx

- **What:** Modal pop-up overlay
- **Data:** `site_popup` table
- **Features:**
 - Shows once per session (if `show_once=true`)
 - Title, body (markdown), image, CTA button
 - Dismiss button
 - Centered on screen

shop/ShopHero.tsx

- **What:** Dynamic, auto-rotating hero at the top of `/shop`
- **Data:** `shop_hero_slides` table (via `getActiveShopHeroSlides()` ; passed in as a prop from the server component)
- **Features:**
 - Image-backed slides (next/image `fill`) with dark gradient + Anton headline + orange CTA pill
 - Auto-rotates (~6s crossfade) with 2+ slides; static with one
 - Pauses on hover/focus; honours `prefers-reduced-motion` (no auto-advance)
 - Keyboard-navigable dot indicators
 - Falls back to the static "The Destiny Store" masthead when no slides are active (handled in `app/shop/page.tsx`)

FloatingSmartSearch.tsx

- **What:** AI-powered conversational search widget (floating morphing pill/circle)
- **Feature:** If `smart_search` service is enabled
- **Behavior:**
 - Click button → pill expands, chat thread opens
 - User types query → full history sent to `/api/chat` (OpenAI, `gpt-4.1-mini`)
 - The route uses **tool-calling** and streams **NDJSON** events (`text` , `tool_result` , `done`). Prose is still parsed for the trailing `OPTION:` / `PAGE:` / `CTA:` lines (clarifying chips + a navigation CTA).
- **Tools** (`lib/smartSearch/tools.ts`): `find_products` (searches published shop products via `getPublishedProducts()` + fuse.js, returns cards), `get_weather` (Open-Meteo, no key), `get_directions` (Google Maps embed), `search_web` (Tavily).
- **Result cards** (`components/smartSearch/ResultCards.tsx`) render below the prose in Smart Search's glass style. The product card offers inline size/colour selection and **add-to-cart** (via `useCart()`), so a visitor can buy without leaving the conversation.
- Chat history + cards stored in component state (not persisted)
- **Optional env vars:** `NEXT_PUBLIC_GOOGLE_MAPS_EMBED_API_KEY` (directions embed — degrades to an "Open in Maps" link without it) and `TAVILY_API_KEY` (web search — degrades to a "not configured" note). Weather and product search need no extra key.

VisualEditOverlay.tsx does not exist. There is no in-page visual editing overlay — it was part of the removed page-builder/Studio feature (migration `20260711_06_remove_page_builder.sql`). Content is now edited through dedicated admin forms (`/admin/posts` , `/admin/store/products/[id]` , etc.), each using **RichTextEditor.tsx** for rich text where applicable.

GlassBloomTracker.tsx

- **What:** Performance tracking script
- **Purpose:** Tracks bloom/glass effect performance for optimization

LiveBanner.tsx

- **What:** "WE ARE LIVE" banner bar, styled like **SiteBanner.tsx** 's bars
- **Data:** **LiveContext** (server-seeded in root layout via `getLiveStatus()` , then polled client-side every 60s)
- **Behavior:** Renders at the top banner slot (offsetting any DB banner below it) whenever the channel is live; hidden on `/live` and `/admin/*` . CTA links to `/live` .

live/LiveExperience.tsx + **live/LivePlayer.tsx**

- **What:** The `/live` page's client UI
- **LiveExperience.tsx** : Switches between live / offline / stream-ended states based on **LiveContext** ; requires the ENDED player event or two consecutive negative polls before dropping out of the live view
- **LivePlayer.tsx** : YouTube IFrame API player with `controls=0` and a fully custom glass control bar (play/pause, mute, volume, fullscreen, live-edge seek) — see `lib/youtubeIframe.ts` for the shared API loader (also used by **SermonPlayer.tsx**)

Page-Specific Components

Kids Ministry (**components/kids/***)

- **KidsHero.tsx** — Hero banner for kids section
- **KidsAges.tsx** — Age groupings (babies, toddlers, kids)
- **KidsProgram.tsx** — Sunday program overview
- **KidsContact.tsx** — Parent contact form

Youth Ministry (**components/youth/***)

- Similar structure to kids

Admin Components (**components/admin/***)

- **AdminSidebar.tsx** — Admin navigation menu
- **AdminHeader.tsx** — Sticky desktop header for the admin shell; shows an "Admin / {section}" breadcrumb (title derived from the pathname) and a "View live site" button
- **RichTextEditor.tsx** — Shared TipTap rich-text editor (HTML output); used by posts, training posts, HR job descriptions, and (since `a22301b`) shop product descriptions. Optional `enableHtmlEmbed` prop adds

a raw-HTML embed block.

Redirects and banner management (`/admin/redirects` , `/admin/banner`) are built inline in their `page.tsx` files rather than as separate reusable components — there is no standalone `RedirectManager.tsx` / `BannerManager.tsx` / `MediaUploader.tsx` / `PageEditor.tsx` / `AdminSermonManager.tsx` . Sermon hiding was removed entirely (migration `20260711_07_remove_sermon_hiding.sql`), and the earlier page-builder/ Studio editor (`PageEditor.tsx` 's likely origin) was removed by `20260711_06_remove_page_builder.sql` .

Admin Content/Training/HR Components (`components/admin/{posts,training,hr}/*`)

- `posts/PostEditor.tsx` — Standalone page editor (uses `RichTextEditor`)
- `training/PostEditor.tsx` — Training post editor (uses `RichTextEditor`)
- `training/CategoryModal.tsx` , `SubgroupModal.tsx` , `FolderModal.tsx` , `IconPicker.tsx` — Training tree CRUD modals
- `hr/HrUI.tsx` — Staff directory, leave requests, documents (main HR dashboard shell)
- `hr/JobModal.tsx` — Create/edit job listing (uses `RichTextEditor`)
- `hr/modals.tsx` — Remaining HR CRUD modals (staff, leave, reviews, applications)

Connect Card (`components/connect-card/*`)

- `ConnectCardCTAs.tsx` — Call-to-action buttons
- The prayer/connection form itself is built inline in `app/connect-card/page.tsx` , not a separate component

Home Page (`components/home/*`)

- `HeroSection.tsx` — Main hero banner with video/image
- `MinistriesGrid.tsx` — Ministry cards (kids, youth, etc.)
- `UpcomingSermons.tsx` — Latest sermons carousel
- `CTAButtons.tsx` — Prominent call-to-action buttons

Shop (`components/shop/*`)

- `ShopProductGrid.tsx` — client wrapper around the `/shop` grid; derives a category filter chip row from the distinct `product.category` values present ("All" + one chip per category, alphabetical), filters the grid client-side on click, and shows an empty state ("Nothing here yet — Try a different category.") when a filter matches nothing. Renders no chips at all if every product shares one/no category.
- `ProductCard.tsx` — storefront product tile (editorial `.shop-card` hover wipe)
- `ProductGallery.tsx` — main image + thumbnail strip (client)
- `ProductBuyPanel.tsx` — colour swatches + size pills + quantity + add-to-basket (client)
- `CartButton.tsx` — header basket icon with live item-count badge (client)
- `CheckoutForm.tsx` — Stripe Express Checkout Element (Apple Pay / Google Pay / Link, shown only when a wallet is available) above the Payment Element card form; both confirm the same `PaymentIntent` (client)

Apple Pay: enabled automatically by `automatic_payment_methods` — no extra API params. The only prerequisite is a **registered payment-method domain**; Stripe hosts the Apple domain-association file, so there is no `/.well-known` file to deploy. Register the live domain(s) once via Stripe Dashboard → Settings → Payment methods → Apple Pay, or run `scripts/register-apple-pay-domain.mjs` with the live `STRIPE_SECRET_KEY` (e.g. `destinytees.uk` and `www.destinytees.uk`). Apple Pay renders on supported devices/browsers only (Safari on Apple hardware).

API Routes & Server Actions

Server Actions vs. API Routes

Server Actions (`app/*/actions.ts`): - Call from client components directly - Typed, type-safe - Auto-serialize arguments & return values - Good for: Form submission, data mutation - Example:

`app/contact/actions.ts`

API Routes (`app/api/*/route.ts`): - HTTP endpoints - Called from `fetch()` or external integrations - Good for: Webhooks, public APIs, external tools - Example: `/api/chat` (Smart Search)

Key Server Actions

`app/contact/actions.ts`

```
"use server";

export async function submitContactForm(data: ContactFormData) {
  // Validate input
  const parsed = contactFormSchema.parse(data);

  // Rate limit: max 5 submissions per IP per hour
  const remaining = await rateLimit(request.ip, "contact-form", 5, 3600);
  if (remaining <= 0) throw new Error("Too many submissions. Try again later.");

  // Insert into database
  const supabase = createServiceClient();
  await supabase.from("contact_messages").insert({
    name: parsed.name,
    email: parsed.email,
    subject: parsed.subject,
    message: parsed.message
  });

  // Send notification email to admin
  await sendEmail({
    to: "hello@destinytees.uk",
    subject: `New contact form: ${parsed.subject}`,
    html: renderEmailTemplate("contactNotification", parsed)
  });

  return { success: true };
}
```

`app/hire/actions.ts`

Similar structure to contact form: - Validate form data - Rate limit - Insert to `hire_enquiries` table - Send email notification

app/connect-card/actions.ts

- Submit prayer request
- Submit connection card (first-time visitor form)
- Inserts to `contact_messages` with category prefix
- Sends email to prayer team

app/jobs/actions.ts

```
export async function applyForJob(jobId: string, formData: ApplicationData) {
  // Validate
  const parsed = applicationSchema.parse(formData);

  // Upload CV to storage
  const file = parsed.cv;
  const { path } = await uploadToStorage(file, `job-cvs/${jobId}`);

  // Insert application
  await supabase.from("job_applications").insert({
    job_id: jobId,
    name: parsed.name,
    email: parsed.email,
    phone: parsed.phone,
    cv_url: path,
    cover_letter: parsed.coverLetter
  });

  // Send confirmation email to applicant
  await sendEmail({
    to: parsed.email,
    subject: "We've received your application",
    html: renderEmailTemplate("applicationReceived", { jobTitle })
  });

  return { success: true };
}
```

Admin API Routes

POST /api/admin/redirects

```
// Create or update redirect
// Body: { slug, targetUrl, label, active }
// Logic:
// 1. Check auth
// 2. Insert/update redirects table
// 3. Trigger full rebuild (redirects are in next.config.ts)
// 4. Return success or error
```

POST /api/admin/popup/upload

```
// Upload image for pop-up
// FormData: { file: File }
// Logic:
// 1. Check auth
// 2. Upload to storage bucket
// 3. Return public URL or signed URL
// 4. Admin uses URL in pop-up form
```

POST /api/admin/revalidate

```
// Manually trigger ISR for a path
// Body: { path: string }
// Logic:
// 1. Check auth
// 2. Call revalidatePath(path) or revalidateTag(tag)
// 3. Return success
```

Public API Routes

POST /api/chat

```
// AI-powered conversational Smart Search (the FloatingSmartSearch widget).
// Body: { messages: { role: "user" | "assistant"; content: string }[] }
// Response: NDJSON stream of { type: "text" | "tool_result" | "done" | "error", ... }

// Logic:
// 1. Rate limit: 20 requests per IP per minute (429 on excess)
// 2. Validate messages: user/assistant roles only (blocks injected system turns),
//    <=2000 chars each, last must be user <=300 chars (else 400)
// 3. getOpenAI() null-check → NDJSON fallback routing to /contact
// 4. Tool-calling loop (up to MAX_TOOL_ROUNDS=3) on gpt-4.1-mini:
//    - System prompt: buildSmartSearchPrompt() (lib/siteKnowledge.ts) – church
//      facts + today's date + tool guidance
//    - tools: TOOL_DEFINITIONS (lib/smartSearch/tools.ts) – find_products,
//      get_weather, get_directions, search_web
//    - Streams assistant text as `text` events; runs tool calls, emits each
//      result as a `tool_result` event, feeds results back, repeats
// 5. Client (FloatingSmartSearch) accumulates `text` (parsed for PAGE/CTA/OPTION)
//    and renders each `tool_result` as a card (ResultCards.tsx)
```

GET /api/youtube/videos

```
// Returns getAllVideos(50) – the sermon archive's video list (lib/youtube.ts).
// Fetched client/server-side by the /sermons page; no separate sync job or
// cache table – YouTube is queried directly on each call.
```

GET /api/youtube/status

```
// { quotaExceeded: boolean } – lib/youtube.ts isYouTubeQuotaExceeded().  
// revalidate = 3600. Lets the client show a friendly fallback if the  
// YouTube Data API daily quota has been used up.
```

GET /api/youtube/thumbnail/[id]

```
// Proxies a YouTube video thumbnail image (avoids hot-linking i.ytimg.com directly).
```

GET /api/youtube/live

```
// Livestream status, polled client-side every 60s by LiveContext  
// Response: { live: boolean, videoId: string | null, title?: string, checkedAt: string }  
// revalidate = 60  
  
// Backed by lib/youtube.ts getLiveStatus() – see Libraries & Utilities.
```

There is no `/api/webhooks/vercel` or GitHub webhook route, and no `youtube-sync` cron/cache job — `app/api/webhooks/` currently only contains `stripe/` (see Shop API below). YouTube data is fetched live on each request, not synced to a cache table.

Shop API

```
// PUBLIC (outside the middleware matcher)
POST /api/store/checkout
// Body: { items: [{ variantId, quantity }], customer: { name, email, phone?, notes? } }
// Recomputes every price + stock from the DB (client prices never trusted),
// creates a pending order + items, creates a Stripe PaymentIntent, returns
// { clientSecret, orderNumber }. Rate-limited per IP.

POST /api/webhooks/stripe
// Stripe signature-verified (STRIPE_WEBHOOK_SECRET), runtime = nodejs, raw body.
// payment_intent.succeeded → order 'paid', decrement variant stock, Resend
// confirmation emails (customer + church). Idempotent. payment_failed → 'cancelled'.

POST /api/store/checkout/bypass
// TEST ONLY. 404 unless server env SHOP_TEST_BYPASS=1. Creates a real order and
// finalises it WITHOUT Stripe (paid, stock decremented, emails) so the full flow
// can be demoed without a payment. The checkout page shows a "Complete test order"
// button when NEXT_PUBLIC_SHOP_TEST_BYPASS=1. Never set either flag in production.

// Shared order logic (pricing recompute, order creation, paid-finalisation) lives
// in lib/checkout.server.ts and is used by checkout, the webhook, and the bypass.

// ADMIN (gated by middleware, site_editor role)
GET|POST      /api/admin/store/products
GET|PUT|DELETE /api/admin/store/products/[id]      // PUT reconciles variants
POST|DELETE   /api/admin/store/products/[id]/images // sharp → WebP → product-images
bucket

GET           /api/admin/store/orders
GET|PATCH    /api/admin/store/orders/[id]      // PATCH updates fulfilment status
GET|POST      /api/admin/shop-hero              // list / create hero slides
PATCH|DELETE /api/admin/shop-hero/[id]          // update (content/active/sort_order)
/ delete

POST          /api/admin/shop-hero/upload        // sharp → WebP → shop-hero-images
bucket
```

Libraries & Utilities

Shop (`lib/shop*.ts` , `lib/stripe.ts` , `lib/cart-store.ts`)

- `lib/shop.ts` — client-safe types (`Product` , `ProductVariant` , `Order` , `CartItem`), `formatPrice(pennies)` , `variantPrice` , `fromPrice` , `totalStock` . Prices are integer pennies (GBP).
 - `lib/shop.server.ts` (`server-only`) — public read fetchers: `getPublishedProducts()` , `getProductBySlug()` , `getAllProductsAdmin()` (via `getSupabaseAdmin()`).
 - `lib/stripe.ts` (`server-only`) — `getStripe()` singleton from `STRIPE_SECRET_KEY` .
 - `lib/cart-store.ts` — `useCart` zustand store persisted to `localStorage` (`destiny-cart`), plus `cartCount` / `cartSubtotal` helpers.
-

lib/youtube.ts

```
// Wrapper around YouTube Data API v3
export async function getAllVideos(maxResults = 50): Promise<YTVideo[]> {
  const youtube = google.youtube({
    version: "v3",
    auth: process.env.YOUTUBE_API_KEY
  });

  // Search for videos in our channel
  const { data } = await youtube.search.list({
    part: "snippet",
    channelId: process.env.YOUTUBE_CHANNEL_ID,
    maxResults,
    order: "date",    // Newest first
    type: "video"
  });

  // Transform to YTVideo type with metadata
  return data.items.map((item) => ({
    id: item.id.videoId,
    title: item.snippet.title,
    description: item.snippet.description,
    thumbnail: item.snippet.thumbnails.high.url,
    publishedAt: item.snippet.publishedAt,
    // ... extract speaker, series from description/title
  }));
}

export async function getLatestVideo(): Promise<YTVideo | null> {
  const videos = await getAllVideos(1);
  return videos[0] ?? null;
}

// Livestream detection – used by the /live page and the "WE ARE LIVE" banner.
// Zero-quota by design: scrapes youtube.com/channel/{id}/live for the
// canonical watch URL + "isLiveNow" flag, falling back to a 1-unit
// videos.list confirm call only when the scrape is ambiguous (never uses
// search?eventType=live, which costs 100 units/call). Fails closed – any
// fetch/parse error or LIVE_DISABLED=1 returns { live: false }.
export async function getLiveStatus(): Promise<LiveStatus> { /* ... */ }
```

Why Supabase Storage isn't used for videos: - YouTube handles CDN/caching automatically - Bandwidth costs for video are lower on YouTube - YouTube playlist/channel management features - Comments, engagement metrics built-in

lib/smartSearch.ts

Parses the LLM response and validates navigation:

```

export interface SmartSearchResult {
  answer: string;           // Main prose answer
  page: string | null;      // Valid page URL (or null)
  ctaLabel: string | null;  // Button text
  options?: string[];       // For clarifying questions
}

export function parseAnswer(raw: string): SmartSearchResult {
  // Extract OPTION: lines (clarifying question choices)
  const options = Array.from(
    raw.matchAll(/^\s*(?:[-*]\s*)?OPTION:\s*(.+?)\s$/gim)
  )
    .map((m) => m[1])
    .slice(0, 4); // Max 4 options

  // Strip OPTION/PAGE/CTA lines from prose
  const prose = raw
    .replace(/^\s*OPTION:\s*.*$/gim, "")
    .replace(/^\s*PAGE:\s*\S+$/gim, "")
    .replace(/^\s*CTA:\s*.*$/gim, "")
    .trim();

  // If options present, this is a clarifying question
  if (options.length >= 2) {
    return { answer: prose, page: null, ctaLabel: null, options };
  }

  // Extract PAGE and CTA from end of response
  const pageMatch = raw.match(/^\s*PAGE:\s*(\S+)\s$/im);
  const ctaMatch = raw.match(/^\s*CTA:\s*(.+?)\s$/im);

  let page: string | null = null;
  if (pageMatch) {
    const candidate = pageMatch[1].replace(/[\.,]"'+$/ , "");
    // Validate against allowlist (prevents hallucinated links)
    if (ALLOWED_PAGES.has(candidate)) page = candidate;
  }

  let ctaLabel: string | null = null;
  if (page && ctaMatch) {
    ctaLabel = ctaMatch[1].trim();
  }

  return { answer: prose, page, ctaLabel };
}

export const FALLBACK_ANSWERS = {
  tooShort: "Ask me anything about Destiny...",
  unavailable: "I can't reach the assistant right now...",
  empty: "I'm not totally sure on that one..."
};

export function cooldownAnswer(): SmartSearchResult {
  // User hit rate limit
  return {
    answer: "You've made a lot of searches in a short time...",
  }
}

```

```

    page: "/contact",
    ctaLabel: "Contact Us"
  };
}

```

Key Security:

1. **Allowlist validation** — Page URLs must be in `PAGE_INTENTS` ; any hallucinated links are dropped
2. **Rate limiting** — Max 5 searches per IP per minute
3. **Fallback answers** — If service unavailable, still show a friendly response (never "error")

`lib/siteKnowledge.ts`

Single source of truth for the AI assistant:

```

export const PAGE_INTENTS = [
  { href: "/give", cta: "Give Now", intent: "giving, donations, bank details..." },
  { href: "/visit", cta: "Plan Your Visit", intent: "visiting, first time..." },
  { href: "/sermons", cta: "Watch Sermons", intent: "sermons, messages..." },
  { href: "/live", cta: "Watch Live", intent: "livestream, live service, Sundays at 11am" },
  { href: "/shop", cta: "Browse Merch", intent: "shop, apparel, merch, buy" },
  { href: "/help", cta: "Help Centre", intent: "help, FAQ, questions" },
  // ... 17 more pages (kids, youth, Alpha, serve, connect, missions, etc.)
];

export const ALLOWED_PAGES = new Set(PAGE_INTENTS.map(p => p.href));

// System prompt sent to OpenAI (truncated here):
const SEARCH_INSTRUCTIONS = `
You are a friendly assistant for Destiny Church Tees Valley.

HOW TO REPLY:
- Write 2-5 conversational sentences
- Never restart the question or use full church name
- Append PAGE: and CTA: tags ONLY if relevant:
  PAGE: /the-path
  CTA: Short Action Label

GROUNDING (MOST IMPORTANT):
- Only state facts in the KNOWLEDGE section below
- Do NOT guess, invent, or assign roles unless listed
- If you don't know, warmly say so and point to /contact

CLARIFYING QUESTIONS:
- If a request is ambiguous, ask ONE short question with 2-4 OPTION: choices
- Do NOT ask follow-ups for simple factual questions

KNOWLEDGE:
... (church-specific facts: pastor names, service times, livestream, shop, mission, etc.) ...
`;

```

Why this pattern?

1. **Prevents hallucination** — Model is grounded in actual church knowledge

2. **Limits links** — Only allows pages we've defined
 3. **Consistent tone** — Friendly, warm, not corporate
 4. **Scalable** — Easy to update facts without retraining
-

`lib/pageContent.ts`

Fetch dynamic content from the `page_content` table:

```
export async function getPageContent(page: string, key: string): Promise<string> {
  const supabase = createServiceClient();
  const { data } = await supabase
    .from("page_content")
    .select("value")
    .eq("page", page)
    .eq("key", key)
    .maybeSingle();
  return data?.value ?? "";
}

// Used by:
export async function getGivingDetails() {
  return Promise.all([
    getPageContent("give", "account_name"),
    getPageContent("give", "sort_code"),
    getPageContent("give", "account_number"),
    getPageContent("give", "text_keyword"),
    getPageContent("give", "text_number")
  ]);
}

export async function getServiceInfo() {
  return Promise.all([
    getPageContent("general", "service_day"),
    getPageContent("general", "service_time"),
    getPageContent("general", "service_address")
  ]);
}
```

Why key-value store?

1. **Editable by non-technical users** — No code changes needed
 2. **Centralized** — All dynamic content in one table
 3. **Cacheable** — Rarely changes; can be cached aggressively
 4. **Typed access** — TypeScript ensures correct key names
-

Note: There is no `lib/roles.ts` in the codebase — the app uses a single-role admin model (any authenticated Supabase user has full `/admin` access), enforced by `middleware.ts` rather than a per-user role table. See Authentication & Authorization below for the actual auth flow.

lib/rateLimit.ts

Per-IP sliding-window rate limiter with escalating cooldowns, **in-memory** (a `Map`, not a database table) — adequate for this site's traffic, per-instance on serverless. Callers namespace keys (e.g. `contact:${ip}`) so endpoints don't trip each other's limits:

```
const WINDOW_MS = 60_000;      // 1-minute sliding window
const MAX_PER_WINDOW = 15;     // requests allowed per window
const COOLDOWN_MS = [5, 10, 20, 40, 60].map((m) => m * 60_000); // escalating cooldowns per offense

const rlStore = new Map<string, { timestamps: number[]; cooldownUntil: number; offenses: number }>();

export function clientIp(source: { headers: Headers } | Headers): string { /* ... */ }
// checkRateLimit(key) reads/writes rlStore, returns { limited, retryAfterMs }
```

// Used by: // Form submissions (contact, hire, apply for job) // `/api/chat` (Smart Search) — separate simpler per-IP counter, 20 req/min // Login attempts (`lib/loginRateLimit.ts` , a related but distinct limiter)

lib/training.ts

Training course management:

```

export interface TrainingCategory {
  id: string;
  name: string;
  description: string;
  icon: string;
  order: number;
}

export interface TrainingModule {
  id: string;
  categoryId: string;
  title: string;
  description: string;
  content: string; // Markdown or HTML
  duration: number; // Minutes
  videoUrl?: string;
  completed: boolean;
}

export async function getTrainingCategories(): Promise<TrainingCategory[]> {
  const supabase = createServiceClient();
  const { data } = await supabase
    .from("training_categories")
    .select("*")
    .order("order");
  return data ?? [];
}

export async function getUserCourseProgress(userId: string) {
  // Fetch completed modules for this user
  // Used to show progress bars and completion badges
}

```

lib/jobs.ts & **lib/hr.ts**

Similar patterns for job listings and HR management.

Authentication & Authorization

Supabase Auth Flow

1. **User signs up/in:** - Email + password sent to Supabase - Returns JWT token - Token stored in secure HTTP-only cookie (via `@supabase/ssr`)
2. **On every request:** - Next.js middleware checks for valid token - If valid, attached to request context - If expired, refresh token is used automatically
3. **On protected routes:** - Server component checks if user exists - Redirects to login if not authenticated
4. **Server actions & API:** - Get user ID from token - Query database with service role (not user role) - Apply RLS policies based on table settings

Authorization Layers

There is no `lib/roles.ts` and no per-section role model — a single authenticated Supabase user has full `/admin` access. Auth is enforced centrally in `middleware.ts`, not in `app/admin/layout.tsx` (which is a client component purely responsible for the sidebar/header shell; it does not check auth itself).

Layer 1: Middleware (`middleware.ts`)

```
// Matcher: "/admin/:path*", "/api/admin/:path*" (+ a "/lite" redirect helper, unrelated to auth)
export async function middleware(request: NextRequest) {
  const { supabase, supabaseResponse } = createClient(request);
  const { data: { user } } = await supabase.auth.getUser();

  if (pathname === "/api/admin/logout") return supabaseResponse; // always reachable
  if (pathname === "/admin/login") return NextResponse.redirect(new URL("/admin", request.url)); // stale bookmark

  // Public admin auth pages must stay reachable while logged out
  if (["/admin/forgot-password", "/admin/reset-password"].some((p) => pathname.startsWith(p))) {
    return supabaseResponse;
  }

  if (!user && pathname.startsWith("/api/admin")) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }
  if (!user) return NextResponse.redirect(new URL("/login", request.url));

  return supabaseResponse;
}
```

Layer 2: Database (RLS)

```
-- Example: Only service role can read HR staff
CREATE POLICY "service only" ON hr_staff USING (false);
```

API routes use the service role key for reads/writes (RLS is bypassed), relying on the middleware gate above rather than per-table role checks.

Why two layers? - Defense in depth — the middleware gate is the single source of truth for "is this visitor allowed into `/admin` at all"; RLS (deny-all + service role) means even a bug in the app layer can't expose sensitive tables to the anon key. - **Simplicity** — no role table to keep in sync; any staff account with a login has full access, which matches how small the admin user base is in practice.

Session Management

- **Session storage:** Secure HTTP-only cookie (managed by `@supabase/ssr`)
 - **Token refresh:** Automatic when expired (hooks in `Providers.tsx`)
 - **Logout:** Clear cookie, redirect to home
 - **CSRF protection:** Next.js built-in (automatic for POST requests)
-

Configuration

`next.config.ts`

```
const nextConfig: NextConfig = {
  // Include ffmpeg binary for serverless functions (if needed)
  outputFileTracingIncludes: {
    "**": ["node_modules/ffmpeg-static/ffmpeg"],
  },

  // Permanent redirects (handled at edge)
  async redirects() {
    return [
      { source: "/prayer-request", destination: "/connect-card", permanent: true },
      { source: "/prayer-requests", destination: "/connect-card", permanent: true }
    ];
  },

  // Security headers
  async headers() {
    return [
      {
        source: "/(.*)",
        headers: [
          {
            key: "Permissions-Policy",
            value: "camera=(), microphone=(), geolocation=()" // Disable dangerous features
          }
        ]
      },
    ],
    // Long-lived cache for static assets (immutable = only update if filename changes)
    {
      source: "/img/:path*",
      headers: [
        { key: "Cache-Control", value: "public, max-age=31536000, immutable" }
      ]
    }
  ],

  // Image optimization
  images: {
    minimumCacheTTL: 2592000, // 30 days
    // Allow images from these domains
    remotePatterns: [
      { protocol: "https", hostname: "i.ytimg.com" }, // YouTube thumbnails
      { protocol: "https", hostname: "storage.buzzsprout.com" }, // Podcast images
      { protocol: "https", hostname: "cdn.churchsuite.com" }, // ChurchSuite events
      { protocol: "https", hostname: "lwmnrbglbtbyypzcenzf.supabase.co" }, // Our Supabase
    ],
    bucket
  },
  // ... more patterns
}
```

```
}  
};
```

tailwind.config.ts

```
import type { Config } from "tailwindcss";  
  
const config: Config = {  
  content: [  
    "./app/**/*..{js,ts,jsx,tsx}",  
    "./components/**/*..{js,ts,jsx,tsx}"  
  ],  
  theme: {  
    extend: {  
      colors: {  
        brand: "#FF6B35", // Church brand color  
        accent: "#004E89",  
        light: "#F7F7F9"  
      },  
      fontFamily: {  
        display: ["var(--font-anton)", // Bold, display headings  
        serif: ["var(--font-playfair)", // Editorial headings  
        sans: ["var(--font-roboto)"] // Body text  
      },  
      animation: {  
        "fade-in": "fadeIn 0.5s ease-in",  
        "slide-up": "slideUp 0.6s ease-out"  
      }  
    }  
  },  
  plugins: []  
};  
  
export default config;
```

tsconfig.json

- `strict: true` — Strict type checking
- `jsx: "react-jsx"` — React 17+ JSX transform
- `@/*` — Path alias for imports (e.g., `@/components/Button`)

`.env.local` Template

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_ROLE_KEY=eyJ...

# YouTube
YOUTUBE_API_KEY=AIZA...
YOUTUBE_CHANNEL_ID=UCxx...

# Email
RESEND_API_KEY=re...

# OpenAI (for Smart Search chat – gpt-4.1-mini, tool-calling)
OPENAI_API_KEY=sk-...

# Smart Search tools (optional – each degrades gracefully without its key)
NEXT_PUBLIC_GOOGLE_MAPS_EMBED_API_KEY=AIZA... # get_directions embed
TAVILY_API_KEY=tvly-... # search_web

# Stripe (shop checkout)
STRIPE_SECRET_KEY=sk_live...
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_live...
STRIPE_WEBHOOK_SECRET=whsec...
SHOP_TEST_BYPASS= # leave unset in production – enables /api/store/checkout/bypass

# GitHub (for CI/CD)
GITHUB_TOKEN=ghp...

# Feature flags (also toggleable via the `service_status` DB table, e.g. 'smart_search')
ENABLE_SMART_SEARCH=true
```

Deployment & Performance

Deployment Target: Vercel

Why Vercel? - **Next.js first-party** — Built by Vercel team - **Edge functions** — Faster redirects, rate limiting - **ISR (Incremental Static Regeneration)** — Cache pages, revalidate on-demand - **CDN** — Automatic global distribution - **Analytics** — Built-in performance monitoring

Caching Strategy

Content	Strategy	TTL	Invalidation
Static assets (<code>/img</code> , <code>/fonts</code>)	Immutable	1 year	Filename change
Home page	ISR	1 hour	<code>revalidatePath("/")</code>
Sermon archive	ISR	4 hours	<code>revalidatePath("/sermons")</code>
Dynamic pages (<code>/[slug]</code>)	ISR	24 hours	<code>revalidatePath("/[slug]")</code>
Redirects	Edge	Infinite	Deploy
API responses	None	-	Fresh on every request
Images	Browser cache	30 days	<code>next/image</code> optimization

Performance Optimizations

1. **Image optimization:** - `next/image` component auto-resizes - WebP format for modern browsers - Lazy loading by default
 2. **Code splitting:** - Dynamic imports for heavy components - Suspense boundaries for loading states
 3. **Database queries:** - No N+1 queries (use `select()` joins) - Cache at Supabase level (pg_cache extension) - Use materialized views for complex queries
 4. **Edge caching:** - Long TTLs on assets - Geography-aware CDN (UK origin)
 5. **Monitoring:** - Vercel Analytics — tracks Web Vitals - Vercel Speed Insights — performance timeline - Custom events logged to analytics
-

Key Design Decisions & Rationale

Decision 1: Service Role Key for Backend

Why: Instead of client tokens accessing database directly:

- **Security** — Client can't be trusted; always use server role for writes
- **RLS consistency** — Service role bypasses RLS; we apply auth in app code
- **Error messages** — Can give user-friendly feedback (vs. RLS "you don't have permission")
- **Audit logging** — API routes log who did what; database doesn't

Decision 2: Supabase for Database

Why: Not a custom Node/Express API:

- **PostgreSQL power** — JSON, full-text search, PostGIS for geo (future)
- **RLS built-in** — Security at database layer
- **Real-time** — Subscriptions for live updates (not currently used)
- **Managed** — No DevOps overhead

Decision 3: Server Components Over Client

Why: Use React server components wherever possible:

- **SEO** — Content rendered server-side, searchable
- **Security** — Secrets stay on server (not in browser)
- **Performance** — Less JavaScript shipped to client
- **Simplicity** — No useState/useEffect for data fetching

Decision 4: AI Knowledge Base Over Retrieval

Why: Single `siteKnowledge.ts` instead of RAG:

- **Reliability** — Model grounded in actual facts; no hallucination risk
- **Control** — Easy to edit facts without retraining
- **Cost** — Fewer API calls (no embedding search)
- **Speed** — Lower latency (fewer round-trips)

Decision 5: TipTap for Rich Text

Why: Not Slate, Draft.js, or Lexical:

- **Smaller bundle** — ~50KB gzipped (vs. 150KB+)
- **Extensions** — Underline, highlight, image, YouTube, placeholder
- **Prosemirror foundation** — Battle-tested
- **No config** — Works out of box

File Manifest

Public Pages

- `app/page.tsx` — Home (hero, latest sermon, CTAs)
- `app/sermons/page.tsx` — Sermon archive (grid, filters)
- `app/sermons/[id]/page.tsx` — Sermon detail (video, transcript)
- `app/live/page.tsx` — Livestream (custom glass player / offline state)
- `app/about/page.tsx` — About church
- `app/beliefs/page.tsx` — Statement of faith
- `app/kids/page.tsx` — Kids ministry
- `app/youth/page.tsx` — Youth ministry
- `app/young-adults/page.tsx` — Young adults
- `app/missions/page.tsx` — Missions partners
- `app/serve/page.tsx` — Volunteer opportunities
- `app/connect/page.tsx` — Connect groups
- `app/give/page.tsx` — Giving & donations
- `app/visit/page.tsx` — Plan a visit
- `app/new-here/page.tsx` — First-time visitor guide
- `app/hire/page.tsx` — Venue hire enquiry
- `app/contact/page.tsx` — Contact form
- `app/connect-card/page.tsx` — Prayer requests
- `app/alpha/page.tsx` — Alpha course
- `app/whats-on/page.tsx` — Events (ChurchSuite embed)
- `app/jobs/page.tsx` — Job listings
- `app/jobs/[slug]/page.tsx` — Job detail
- `app/shop/page.tsx` — Store (product grid)
- `app/shop/[slug]/page.tsx` — Product detail (variants, gallery)
- `app/shop/cart/page.tsx` — Shopping cart (Zustand + localStorage)
- `app/shop/checkout/page.tsx` — Stripe Payment Element checkout
- `app/shop/checkout/success/page.tsx` — Order confirmation
- `app/baptism/page.tsx` — Baptism information & registration
- `app/child-dedication/page.tsx` — Child dedication requests
- `app/safeguarding/page.tsx` — Safeguarding & child protection
- `app/help/page.tsx` — Help centre & FAQ
- `app/training/page.tsx` — /training resource library
- `app/volunteer/page.tsx` , `app/links/page.tsx` , `app/destiny-recovery/page.tsx` , `app/dckids/page.tsx` — Volunteer sign-up, next-steps links, recovery info, kids camp campaign
- `app/accessibility/page.tsx` , `app/privacy/page.tsx` , `app/terms/page.tsx` , `app/data-gdpr/page.tsx` — Preferences & legal pages

- `app/annual-report-2025/page.tsx` — Annual report campaign page
- `app/[slug]/page.tsx` — Dynamic catchall (posts table)

Admin Pages

- `app/login/page.tsx` — Staff sign-in (there is no `app/admin/login` ; that path is a stale-bookmark redirect to `/admin`)
- `app/admin/page.tsx` — Admin home (dashboard)
- `app/admin/banner/page.tsx` — Banner management
- `app/admin/popup/page.tsx` — Pop-up management
- `app/admin/redirects/page.tsx` — Redirect management
- `app/admin/cache/page.tsx` — Cache invalidation
- `app/admin/alpha/page.tsx` , `app/admin/bible-course/page.tsx` , `app/admin/recovery/page.tsx` — Course event management
- `app/admin/featured-course/page.tsx` — What's On featured course picker
- `app/admin/store/page.tsx` — Store management
- `app/admin/store/products/new/page.tsx` — Create product
- `app/admin/store/products/[id]/page.tsx` — Edit product (variants, stock)
- `app/admin/store/orders/page.tsx` — Orders list
- `app/admin/store/orders/[id]/page.tsx` — Order detail (fulfillment)
- `app/admin/store/hero/page.tsx` — Shop hero slides (add/edit/reorder rotating hero)

Component Directory

- **~120 components** organized by feature:
- Global: Header, Footer, Providers, CookieBanner, Analytics, FloatingSmartSearch
- Admin: Sidebar, AdminHeader, RichTextEditor (shared editor), posts/training/hr editors
- Ministry-specific: Kids, Youth, Young Adults, Alpha
- Shop: ProductCard, ShopProductGrid, ShopHero, cart, checkout
- Forms: ConnectCard, ContactForm, HireForm
- Content: Training, Jobs, HR

Libraries (`lib/`)

- **~40 utility files** including:
- Supabase clients (admin, browser)
- API wrappers (YouTube, OpenAI, Resend, Stripe)
- Domain logic (sermons, jobs, HR, training, shop)
- Email templates
- Rate limiting (in-memory), auth is handled by `middleware.ts` (no `roles.ts`)
- Feature flags (`serviceStatus.ts`)

API Routes (`app/api/`)

- **Admin endpoints:** Banners, redirects, pop-ups, cache revalidation, posts, training, alpha-events, featured-course, HR, store management
- **Public endpoints:** `/api/chat` (Smart Search tool-calling chat), YouTube (videos/status/thumbnail/live), Alpha info, training unlock
- **Store endpoints:** Stripe checkout + Payment Element, order management
- **Webhooks:** Stripe only (`/api/webhooks/stripe`) — no GitHub or Vercel deployment webhook route

Database Migrations

- **34 migration files** defining schema for:
 - URL redirects, hidden videos (removed), site content (banners, pop-ups, page_content)
 - Event management (Alpha course, Bible Course, Recovery, featured course)
 - HR management (staff, leave, documents, reviews)
 - Job listings & applications
 - Feature flags, staff login audit
 - Training resource library, standalone posts
 - Shop (products, variants, orders, hero slides) and RLS/security hardening passes
-

Summary

Destiny Church Tees Valley is a **professional, full-stack Next.js application** that combines:

1. **Public-facing website** — Sermons, ministries, contact, events, livestream
2. **Member engagement** — Prayer requests, volunteering, training, small groups
3. **Admin dashboard** — Content management, HR tools, store management
4. **Ecommerce** — Online store with Stripe payments, product variants, order management
5. **Intelligent features** — Smart Search tool-calling chat assistant, live banner management

Architecture: Server-driven React with Supabase PostgreSQL, deployed to Vercel with ISR caching. All sensitive operations use service-role API proxies with explicit auth checks. Code is type-safe (TypeScript), styled with Tailwind CSS, and tested with Playwright E2E.

Key Differentiators: - **No vendor lock-in** — Open-source stack (Next.js, React, Tailwind, PostgreSQL) - **Security first** — RLS, rate limiting, CSRF protection, signed URLs - **AI-ready** — Grounded knowledge base ([siteKnowledge.ts](#)) powering Smart Search's tool-calling chat (products, weather, directions, web search) - **Accessible** — Semantic HTML, ARIA labels, keyboard nav - **Mobile-first** — Responsive design, tested on real devices - **Performant** — ISR caching, image optimization, edge functions

This repository serves as a **reusable platform** for churches nationwide, licensed through Square Media Group.

Document Version: 1.0.2

Created: June 18, 2026

For: Destiny Church Tees Valley

By: Square Media Group (Malachi malachi@squaremediagroup.org)